

MAGE: Conjuring Boolean Masking Gadgets with Large Language Models for Side-Channel Leakage Reduction

Tasneem Suha, Tanzim Mahfuz, and Prabuddha Chakraborty

Department of Electrical & Computer Engineering, University of Maine, Orono, ME, USA
{tasneem.suha, tanzim.mahfuz, prabuddha}@maine.edu

Disclaimer: This report was prepared as an account of work sponsored by an agency of the United States Government. Neither the United States Government nor any agency thereof, nor any of their employees, makes any warranty, express or implied, or assumes any legal liability or responsibility for the accuracy, completeness, or usefulness of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately owned rights. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States Government or any agency thereof. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States Government or any agency thereof.

Abstract—Power side-channel attacks recover secret data from the physical activity of a hardware implementation, even when the algorithm is mathematically secure. Boolean masking provides defense against this by splitting every sensitive value into shares, but it depends on nonlinear gadgets that must limit leakage without extreme randomness, area, or power. Most masking design process retrieve these gadgets from a small fixed library of hand-crafted constructions such as Trichina’s masked AND. A fixed library is assigned to every design to one gate structure and randomness cost, and prevents the many functionally equivalent realizations that may leak less or synthesize more efficiently.

We present MAGE, a framework that generates masking gadgets automatically instead of drawing them from a fixed set. A large language model proposes diverse output-share equations for AND, OR, NAND, and NOR gates, and an exhaustive checker retains only those that are functionally correct and non-duplicate. The verified equations are compiled into structural gate-level libraries and inserted at locations chosen by measurement-based or structure-based placement, enabling matched-location comparison against a Trichina baseline. Across nine benchmarks, the generated gadgets reduce whole-design TVLA leakage more than Trichina at every location, lowering average leakage by 51.7% against 26.3% under the stronger placement. We frame this as an empirical leakage-structure result. MAGE verifies functional correctness, not probing security, and share-preserving composition remains future work.

Index Terms—Side-channel attacks, Boolean masking, large language models, hardware security

I. INTRODUCTION

Power side-channel attacks recover sensitive values by analyzing the power variations of a hardware implementation, even when the underlying algorithm is mathematically secure. This threat is acute for field-deployed cyber-physical systems such as smart grid infrastructure, where devices are physically accessible and a single key recovery can compromise metering, control, or billing across the network. Boolean masking counters this by splitting each sensitive value into multiple shares

processed without reconstructing the original. Designing the nonlinear gadgets a masked circuit requires remains difficult, since they must preserve functionality while limiting leakage, glitches, randomness cost, and overhead [1], [2].

Most masking flows depend on a small library of predefined gadgets. Automated methods such as AGEMA and AGMNC build masked hardware from known-secure components or nonlinear specifications [3], [4], verification tools such as SILVER and PROLEAD check independence or probing security [5], [6], and VALIANT and POLARIS decide where a countermeasure is applied [7], [8]. Across these, transformation choices remain fixed by the gadget library. Generative AI can explore broader Boolean decompositions: an LLM proposes output-share equations with different partial products and random variables. But these equations cannot be trusted on syntax alone, as they may be incomplete, incorrect, or unsynthesizable, so an LLM-based flow must separate stochastic generation from deterministic checking.

We introduce **MAGE** (Masking-gadget Automated Generation Engine), a two-module framework for generating and evaluating masking transformations. Module I uses an LLM to generate candidate output-share equations for AND, OR, NAND, and NOR gates, exhaustively evaluates each over its input-share and random variables, and rejects incorrect or duplicate candidates. Surviving equations become structural logic-cell architectures, ranked by cell count and estimated leakage reduction into libraries CATEGORY 1 and CATEGORY 2, the top-ranked and second-ranked gadget per gate type. Module II places these architectures using VALIANT, which selects gates whose measured TVLA leakage exceeds a threshold, or POLARIS, which ranks gates by local structural features. For each strategy, the same locations are used to compare CATEGORY 1, CATEGORY 2, and the TRICHINA baseline, and the transformed designs are evaluated with TVLA and, when enabled, Synopsys Design

Compiler for area, power, and delay. The main contributions of this work are as follows:

- **LLM-driven candidate generation:** We develop a flow that uses an LLM to produce Boolean output-share equations and controls acceptance through exhaustive functional and uniqueness verification.
- **Automated architecture construction:** We provide a process that converts accepted equations into reusable structural gate-level masking libraries.
- **Placement integration and matched comparison:** We integrate the generated libraries with both measurement-based VALIANT placement and structure-based POLARIS placement, enabling matched-location comparisons against a manually designed baseline.
- **Leakage and overhead evaluation:** We evaluate the generated architectures across nine benchmarks using target-level and whole-design leakage metrics together with area, power, and delay overhead. The generated libraries reduce leakage more than TRICHINA at every placement, and we report their power, area, and delay costs under the same synthesis settings.

This work targets individual gate transformations. Subgraph-level generation, fresh randomness, share-preserving integration, and robust-probing verification remain future work, and keep functional correctness and empirical TVLA gains from standing in for a formal masking-security result.

II. MOTIVATION, BACKGROUND, AND RELATED WORKS

Masking commonly relies on fixed, hand-crafted constructions, such as Trichina’s masked AND, reused across designs and technology libraries. This motivates a systematic approach for generating masking schemes tailored to the target.

Beyond a Fixed Masking Construction: The same masked function can be realized using different gate structures, share-routing patterns, logic depths, and amounts of fresh randomness. Trichina commits to one such structure, whereas our approach uses an LLM to generate and explore structurally diverse candidates. Functional and security checks remove invalid constructions before implementation.

Balancing Leakage Reduction and PPA Overhead: These structural differences lead to different leakage and PPA overheads after synthesis. We therefore evaluate valid candidates using the target cell library and search for greater leakage reduction while satisfying the specified PPA constraints. Our framework automates this instead of relying on a single predefined masking structure.

Masking and Trichina: Boolean masking splits a sensitive variable x into $d + 1$ shares such that $x = \bigoplus_{i=1}^{d+1} x^{(i)}$, thereby randomizing the intermediate values processed by the circuit [9]. Trichina introduced compact masked constructions for nonlinear operations, including AND and OR gates, by combining masked inputs with fresh randomness [1].

TABLE I
SCOPE OF **MAGE** WITH STATE-OF-THE-ART STUDIES.

Work	Auto.	LLM	Check	Place	D-Eval.
Trichina [1]	–	–	–	–	–
AGEMA [3]	✓	–	✓	–	–
AGMNC [4]	✓	–	✓	–	–
Netlist Whisperer [11]	✓	✓	–	–	–
EA gadgets [12]	✓	–	✓	–	–
VALIANT [7]	–	–	–	✓	✓
POLARIS [8]	–	–	–	✓	✓
MAGE	✓	✓	✓	✓	✓

Auto.: automated masking construction; Check: deterministic functional/security checking; Place: design-level placement; D-Eval.: joint design-level leakage & PPA evaluation.

TVLA: Test Vector Leakage Assessment (TVLA) detects data-dependent leakage by applying Welch’s t -test to two sets of power traces, $\mathcal{Q}_0$ and $\mathcal{Q}_1$ [10]:

$$t = \frac{\mu_0 - \mu_1}{\sqrt{\frac{s_0^2}{n_0} + \frac{s_1^2}{n_1}}} \quad (1)$$

A value of $|t| > 4.5$ is commonly treated as evidence of detectable leakage. Conversely, remaining within this threshold indicates that no leakage was detected under the tested conditions, not a formal proof of security.

VALIANT and POLARIS: VALIANT [7] performs net-level TVLA on a gate-level netlist using technology-library information, then selectively protects leaky nets through a lightweight hybrid masking strategy. POLARIS [8] replaces this fixed selection process with an XAI-guided model trained from synthetically generated masking samples and structural gate features. TVLA is used to label training samples and evaluate the final design, while model inference ranks masking locations without rerunning TVLA during gate selection.

Related Work: Early masking techniques relied on manually derived constructions, with Trichina providing a compact implementation for nonlinear operations [1]. Subsequent work automated different parts of masked-hardware development. AGEMA transforms unprotected circuits using composable masking gadgets [3], while AGMNC constructs masked nonlinear components from lookup-table specifications and validates them using SILVER [4]. Evolutionary search has also been used to optimize robust-probing-compliant gadgets for physical-design objectives [12]. AI-assisted approaches address complementary stages. Netlist Whisperer uses fine-tuned LLMs for leakage classification and generation of DOM-protected HDL [11]. VALIANT identifies countermeasure locations through TVLA, whereas POLARIS uses an explainable machine-learning model to rank locations from structural features [7], [8]. MAGE uses a general-purpose LLM to explore masking equations, deterministically checking functionality and uniqueness, constructing reusable gate-level libraries, and evaluating them through matched placement, TVLA, and PPA analysis. Table I compares the scope of these methods rather than their formal security strength.

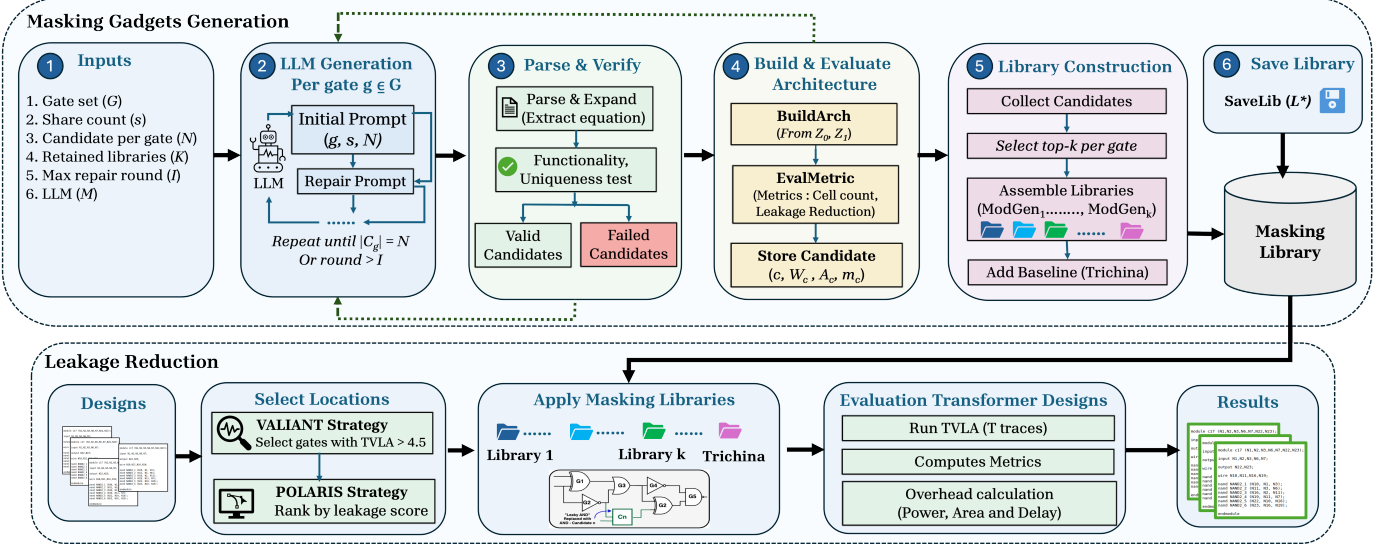


Fig. 1. Overview of the **MAGE** methodology.

III. METHODOLOGY

A. Framework Overview

Figure 1 presents the overall workflow of the **MAGE** framework, while Algorithm 1 summarizes its execution. The framework consists of two modules. Module I uses a large language model (LLM) to generate masking equations, verifies their functional correctness and uniqueness, converts the accepted equations into gate-level architectures, and constructs a reusable masking library. Module II selects transformation locations using VALIANT or POLARIS, inserts the corresponding architectures, and evaluates the transformed designs in terms of leakage reduction and implementation overhead. The algorithm takes as input the target gate set G , share count s , requested candidate count N_c , retained library count K , maximum repair rounds I , LLM $\mathcal{M}$, ranking-design set $\mathbb{D}_r$, evaluation-design set $\mathbb{D}$, placement strategies $\mathbb{P}$, technology library $\mathcal{T}$, TVLA trace count T , leakage threshold τ , transformation budget B , and PPA-evaluation flag η . It returns the masking library $\mathcal{L}^*$ and the design-level evaluation results $\mathbb{R}$.

B. LLM-Based Masking Generation

Module I corresponds to Lines 1–22 of Algorithm 1. It generates candidate masking equations for each target gate, verifies the generated candidates, evaluates them using a configurable ranking-design set, and constructs the final masking library. The current implementation considers

$$G = \{\text{AND}, \text{OR}, \text{NAND}, \text{NOR}\} \quad (2)$$

1) *Candidate Generation and Repair*: Lines 1–5 initialize the candidate-generation process. The set $\mathcal{V}$ collects the verified candidates for all gate types. For each gate $g \in G$, the framework creates an accepted candidate set $\mathcal{C}_g$, a rejected

candidate set $\mathcal{F}_g$, and a generation counter r . The loop in Line 6 continues until either N_c valid candidates are obtained or the repair budget I is exhausted. During the first round, Lines 7–8 use **Prompt** to prepare an LLM request based on the target gate g , share count s , and requested candidate count N_c . For the two-share case, the prompt requests complete equations for Z_0 and Z_1 using the input shares A_0, A_1, B_0 , and B_1 , together with random variables such as r_1 and r_2 .

If the initial response does not provide enough valid candidates, Lines 9–10 use **RepairPrompt** to prepare another request. The repair prompt includes the previously accepted and rejected candidates, helping the LLM avoid repeated or incorrect equations. Line 11 submits the prompt π to the LLM $\mathcal{M}$ and stores the response in $\mathcal{R}_g$. Line 14 updates the generation counter after each request.

2) *Response Parsing and Expansion*: Line 12 uses **Parse** to separate the LLM response into individual candidates and expand intermediate variables into complete Boolean expressions. For example, $p_0 = A_0 \wedge B_0$ and $Z_0 = p_0 \oplus r_1$, is expanded to $Z_0 = (A_0 \wedge B_0) \oplus r_1$. The parsed candidates are stored in $\mathcal{P}_g$. This step only extracts and organizes the generated equations. Candidates with missing or invalid expressions are rejected during verification and recorded in $\mathcal{F}_g$.

3) *Functional and Uniqueness Verification*: Line 13 uses **Verify** to check the newly parsed candidates together with those already accepted. For a two-share candidate $c = (Z_0, Z_1)$, the verifier confirms that $Z_0 \oplus Z_1 = f_g(A_0 \oplus A_1, B_0 \oplus B_1)$ for every possible assignment of the input shares and random variables. Candidates that fail the functional test or duplicate an existing candidate are stored in $\mathcal{F}_g$. Functionally correct and unique candidates are added to $\mathcal{C}_g$. This step verifies the correctness and uniqueness of the generated equations.

Algorithm 1: Masking Generation & Evaluation

Input: $G, s, N_c, K, I, \mathcal{M}, \mathbb{D}_r, \mathbb{D}, \mathbb{P}, \mathcal{T}, T, \tau, B, \eta$ **Output:** $\mathcal{L}^*, \mathbb{R}$

```
1  $\mathcal{V} \leftarrow \emptyset;$ 
2 foreach  $g \in G$  do
3    $\mathcal{C}_g \leftarrow \emptyset;$ 
4    $\mathcal{F}_g \leftarrow \emptyset;$ 
5    $r \leftarrow 0;$ 
6   while  $|\mathcal{C}_g| < N_c \wedge r \leq I$  do
7     if  $r = 0$  then
8        $\pi \leftarrow \text{Prompt}(g, s, N_c);$ 
9     else
10       $\pi \leftarrow \text{RepairPrompt}(g, s, N - |\mathcal{C}_g|, \mathcal{C}_g, \mathcal{F}_g);$ 
11       $\mathcal{R}_g \leftarrow \mathcal{M}(\pi);$ 
12       $\mathcal{P}_g \leftarrow \text{Parse}(\mathcal{R}_g);$ 
13       $(\mathcal{C}_g, \mathcal{F}_g) \leftarrow \text{Verify}(g, \mathcal{C}_g \cup \mathcal{P}_g, s);$ 
14       $r \leftarrow r + 1;$ 
15   foreach  $c \in \mathcal{C}_g$  do
16      $(\mathcal{W}_c, \mathcal{A}_c) \leftarrow \text{BuildArch}(c.Z_0, c.Z_1);$ 
17      $q_c \leftarrow \text{RankCost}(\mathcal{A}_c, g, \mathbb{D}_r, \mathcal{T});$ 
18      $\mathcal{C}_g[c] \leftarrow (c, \mathcal{W}_c, \mathcal{A}_c, q_c);$ 
19    $\mathcal{V}[g] \leftarrow \mathcal{C}_g;$ 
20  $\mathcal{L}^* \leftarrow \text{SelectLib}(\mathcal{V}, K);$ 
21  $\mathcal{L}^*[\text{TRICHINA}] \leftarrow \text{Trichina}();$ 
22  $\text{SaveLib}(\mathcal{L}^*);$ 
23  $\mathbb{R} \leftarrow \text{TransformEval}(\mathbb{D}, \mathcal{L}^*, \mathbb{P}, \mathcal{T}, T, \tau, B, \eta);$ 
24 return  $(\mathcal{L}^*, \mathbb{R});$ 
```

4) Architecture Construction and Candidate Ranking:

Lines 15–18 process each verified candidate $c \in \mathcal{C}_g$. Line 16 uses **BuildArch** to convert the output-share equations into a structural gate-level architecture. The function produces the required wire declarations $\mathcal{W}_c$ and logic-cell instances $\mathcal{A}_c$, assigning unique names to intermediate signals. Line 17 uses **RankCost** to evaluate the candidate on the configurable ranking-design set $\mathbb{D}_r$. The candidate architecture is applied to the compatible gate locations of each ranking design and mapped using the technology library $\mathcal{T}$. The resulting ranking cost q_c represents the average mapped cell count across the ranking designs. Candidates with lower q_c values are preferred. The ranking-design set is used only during candidate selection. It is separate from the design set $\mathbb{D}$ used for the final leakage and PPA evaluation. This allows other small gate-level designs to be used for ranking without changing the remaining framework. Line 18 stores the candidate equations, generated wires, architecture, and ranking cost in $\mathcal{C}_g$. Line 19 then adds the evaluated candidates for gate g to $\mathcal{V}$.

5) *Masking Library Construction:* After all target gates have been processed, Line 20 uses **SelectLib** to construct up to K generated masking libraries. Candidates are ranked independently for each gate type using q_c . The highest-ranked

candidate for each gate type forms CATEGORY 1, while the second-ranked candidate forms CATEGORY 2. The k -th masking library uses the k -th ranked candidate from each gate type. Line 21 adds TRICHINA as the baseline, and Line 22 stores the final library $\mathcal{L}^*$ for design transformation.

C. Leakage-Aware Design Transformation

Module II corresponds to Line 23 of Algorithm 1. The **TransformEval** function applies the masking libraries to the designs and measures their leakage and implementation cost. It receives the design set $\mathbb{D}$, masking library $\mathcal{L}^*$, placement strategies $\mathbb{P}$, technology library $\mathcal{T}$, trace count T , leakage threshold τ , transformation budget B , and PPA flag η .

1) *Netlist Parsing and Baseline Leakage Profiling:* For each design in $\mathbb{D}$, **TransformEval** first parses the structural Verilog netlist and identifies the maskable AND, OR, NAND, and NOR gates. It also extracts the gate connections and fanout relationships required by the placement strategies. The original design is then evaluated using test vector leakage assessment (TVLA). The leakage value of gate g is represented by $\ell_g = |t_g|$, where t_g is the Welch t -statistic. A gate is classified as leaky when $\ell_g > \tau$. The resulting baseline leakage map is generated once for each design and reused when comparing all masking libraries.

2) *Transformation-Location Selection:* After baseline profiling, **TransformEval** selects the gates to be transformed using either VALIANT or POLARIS. Under VALIANT, the framework selects maskable gates whose measured TVLA values exceed the threshold τ . Under POLARIS, each maskable gate is represented using its local structural features. The trained POLARIS model assigns a leakage-risk score to each gate and selects the top B locations. When an explicit POLARIS budget is not provided, B is matched to the number of gates selected by VALIANT. For each placement strategy, the same selected locations are reused for CATEGORY 1, CATEGORY 2, and TRICHINA. This ensures that the masking constructions are compared under the same transformation budget and at identical locations.

3) *Architecture Injection:* For each placement strategy and masking library, the original netlist is copied and every selected gate is replaced by the corresponding gate-specific architecture from $\mathcal{L}^*$. For example, a selected AND gate is replaced by the AND architecture from CATEGORY 1, CATEGORY 2, or TRICHINA. The injector assigns unique names to all newly introduced wires and cell instances to avoid naming conflicts. It also records which cells were introduced for each original target gate, allowing their post-transformation leakage to be associated with that gate.

4) *Post-Transformation Leakage Evaluation:* After architecture injection, the transformed design is evaluated using the same TVLA settings and trace count T as the original design. The post-transformation leakage of each target gate is obtained from the cells introduced by its replacement architecture. The

solved-gate rate reports the percentage of originally leaky target gates whose leakage falls below τ after transformation. The target-level leakage reduction compares the total leakage of the selected gates before and after transformation. The framework also reports whole-design leakage reduction by considering transformed and unchanged nodes. All leakage measurements produced for the evaluated masking libraries and placement strategies are stored in $\mathbb{R}$.

5) *PPA Evaluation*: When $\eta = 1$, *TransformEval* evaluates the original and transformed netlists using Synopsys Design Compiler and the technology library $\mathcal{T}$. The flow reports area, power, and critical-path delay for each design. For a metric $x \in \{A, P, D\}$, the percentage overhead is computed as

$$\Delta x = 100 \frac{x_{\text{mask}} - x_{\text{orig}}}{x_{\text{orig}}}, \quad (3)$$

where x_{orig} and x_{mask} are the measurements of the original and transformed designs, respectively. After all designs, placement strategies, and masking libraries have been evaluated, the leakage and PPA results are stored in $\mathbb{R}$. Finally, Line 24 returns the masking library $\mathcal{L}^*$ and the complete result set $\mathbb{R}$.

IV. RESULTS

A. Experimental Setup

We evaluate the framework using the target gate set $G = \{\text{AND}, \text{OR}, \text{NAND}, \text{NOR}\}$ and two-share masking, i.e., $s = 2$. For each gate type, the LLM is asked to generate $N = 4$ candidates, with at most $I = 3$ repair rounds. The generation backend $\mathcal{M}$ is GPT-5 mini. The best and second-best candidates are retained, giving $K = 2$ generated masking libraries, denoted CATEGORY 1 and CATEGORY 2. The candidate-ranking set, denoted by $\mathbb{D}_r$, contains the ISCAS'85 c3540 and c5315 circuits [13]. Functionally valid candidates are inserted into the compatible gate locations of these designs and mapped using the technology library $\mathcal{T}$, which is the GSCL 45 nm standard-cell library in our experiments. Candidates are ranked independently for each gate type according to their mapped cell-count cost. The ranking designs in $\mathbb{D}_r$ are used only for candidate selection and are not included in the final leakage evaluation. The final evaluation set, denoted by $\mathbb{D}$, contains nine gate-level benchmarks: arbiter, sin, voter, square, sqrt, div, memctrl, multiplier, and log2. These designs are obtained from the EPFL and MIT CEP benchmark collections [14], [15]. The placement-strategy set is $\mathbb{P} = \{\text{VALIANT}, \text{POLARIS}\}$. Leakage is evaluated using $T = 10,000$ traces, and the leakage threshold is set to $\tau = 4.5$. For POLARIS, we use the AdaBoost model provided by the original implementation with structural locality 7. The transformation budget B is matched across masking constructions so that CATEGORY 1, CATEGORY 2, and TRICHINA are evaluated at the same number of target locations under a given placement strategy. The PPA flag η enables area, power, and critical-path delay evaluation using

TABLE II
TOTAL LEAKAGE REDUCTION (%), $\uparrow$ IS BETTER, BEST RESULT PER DESIGN IN BOLD. POL = POLARIS; VAL = VALIANT.

Design	TRICHINA		CATEGORY 1		CATEGORY 2	
	VAL	POL	VAL	POL	VAL	POL
arbiter	19.32	21.93	26.58	55.29	23.65	53.33
sin	26.30	25.26	47.80	68.71	43.29	62.02
voter	29.38	26.94	48.28	49.77	48.93	50.70
square	21.59	18.88	35.27	43.35	43.02	40.74
sqrt	23.17	29.19	57.21	64.83	42.85	36.96
div	20.71	23.67	29.45	39.85	41.58	34.50
memctrl	19.76	21.00	15.48	43.80	29.49	37.21
multiplier	28.17	31.02	39.47	45.18	41.45	44.12
log2	33.15	38.62	54.24	54.76	52.64	55.14
Average	24.62	26.28	39.30	51.73	40.77	46.08

Synopsys Design Compiler. Table II, Figures 2 and 3 experiments are executed on a system with an Intel Core i9-11900 processor and 32 GB of memory where Table III experiment is made on Intel Xeon W9-3495X CPU, 512 GB RAM, and four NVIDIA RTX 6000 Ada GPUs, each with 48 GB VRAM.

B. Leakage Reduction Comparison: Categories vs Trichina

Using TVLA with $T = 10,000$ traces and a leaking threshold of $|\tau| > 4.5$, we report the percentage reduction in aggregate whole-design leakage after masking, evaluated under both placement policies: VALIANT selects gates by measured TVLA, while POLARIS predicts locations from netlist structure. Table II shows the generated constructions consistently improve on Trichina. Under VALIANT, average reduction rises from 24.62% for Trichina to 39.30% and 40.77% for CATEGORY 1 and CATEGORY 2; under POLARIS the gain is larger, 51.73% and 46.08% against 26.28%. The best configuration thus improves the average by 16.15 percentage points under VALIANT and 25.45 under POLARIS. A generated construction performs best on every benchmark. Under POLARIS, CATEGORY 1 leads on six designs, while CATEGORY 2 leads on voter, div, and log2. Under VALIANT, CATEGORY 2 outperforms Trichina on all nine designs and CATEGORY 1 on eight, except memctrl. POLARIS improves Trichina by 1.66 points, compared with 12.43 and 5.31 points for the generated libraries.

C. PPA Overhead Comparison

We evaluate area, power, and delay on six benchmarks, arbiter, sin, voter, square, sqrt, and div, protecting the same POLARIS-selected gates with Trichina, CATEGORY 1, and CATEGORY 2 and reporting relative to the unmasked netlist. Figure 2 shows comparable area across schemes, averaging 227%, 230%, and 226%, indicating that area is governed by the number of protected operations rather than gadget choice. Delay favors the generated constructions, with CATEGORY 2 lowest at 129% against 150% for Trichina.

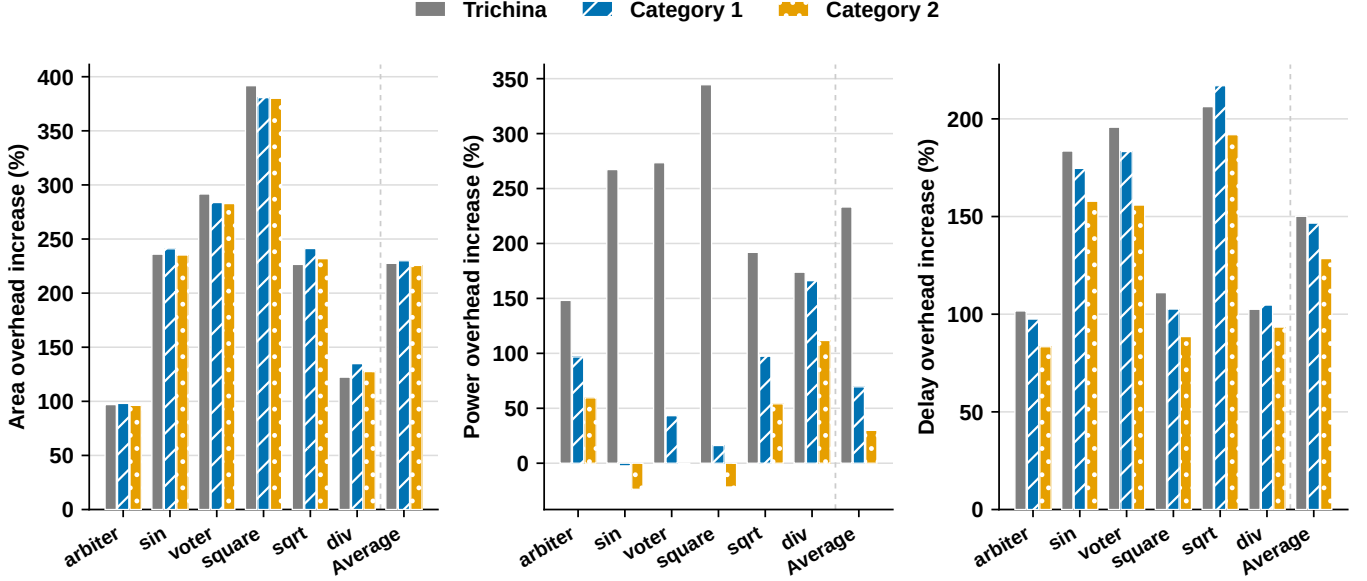


Fig. 2. Area, power, and delay overhead of each masking scheme relative to the unmasked design, per benchmark and averaged.

The largest difference is power. Trichina averages 233%, while CATEGORY 1 and CATEGORY 2 reach 70% and 30%, reducing the added power cost by roughly 70% and 87% and staying lower than Trichina on every benchmark. For *sin* and *square*, CATEGORY 2 shows slightly negative estimated overhead, indicating that synthesis optimizations offset the added logic under the assumed activity model. These are mapped-netlist estimates under fixed synthesis assumptions, not measured silicon power.

D. Robustness Across Independent Generations

Because generation is non-deterministic, the same request can return structurally different but functionally equivalent, independently verified constructions, whereas Trichina is a single hand-derived netlist that synthesizes identically every time. Since results elsewhere rest on whichever candidate the pipeline froze, we ask whether they describe the method or one favorable draw. Holding placement fixed to POLARIS and the benchmark to *square.v*, we ran generation independently $R = 15$ times, froze the top candidate per category each run, and recorded the percentage of leaky nodes POLARIS resolved. Figure 3 plots the distributions against Trichina’s fixed value. Both categories clear the baseline in all fifteen draws, and the distributions are narrow relative to their separation from Trichina. On this benchmark and placement, the advantage over Trichina is stable across independent generations rather than an artifact of a single favorable draw.

E. Changing LLM Backends

Table III compares five backends under the same flow. Valid is the percentage of candidates passing functional and

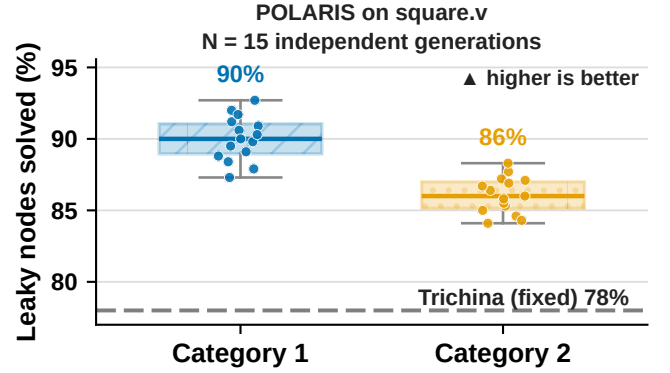


Fig. 3. Leaky nodes solved across $N = 15$ independent generations (box: interquartile range and median; points: individual generations; dashed: Trichina’s fixed result).

TABLE III
EFFECT OF VARIOUS LLM BACKENDS.

Model	Valid (%)	Cells	Leak. red. (%)	Cost
GPT-5-mini	95.2	44	68.8	0.09
Gemini-3-Flash	100	49	41.1	0.31
Claude-Opus-4.8	100	53	73.9	0.23
Qwen3-30B	65.2	66	30.8	–
DeepSeek-R1-70B	65.2	49	43.2	–

uniqueness checking; Cells is the mapped cell count of the selected library across the four gate types. Backend choice affects validity, cost, and leakage reduction independently. Claude-Opus-4.8 gives the strongest reduction, 73.9% at 100% validity, while Gemini-3-Flash matches 100% validity with a

smaller 49-cell library but only 41.1% reduction, so valid, low-cost generation does not guarantee the strongest masking. Among open-weight backends, DeepSeek-R1-70B beats Qwen3-30B, 43.2% versus 30.8%, at equal 65.2% validity. Invalid constructions are filtered before reaching the netlist.

V. CONCLUSION

We presented MAGE, a framework that generates masking gadgets with a large language model instead of drawing them from a fixed library. An LLM proposes diverse output-share equations for AND, OR, NAND, and NOR gates, an exhaustive checker retains only those that are functionally correct and non-duplicate, and the survivors are compiled into gate-level libraries and inserted by measurement-based or structure-based placement. Across nine benchmarks, the generated gadgets reduce whole-design TVLA leakage more than Trichina at every location, lowering average leakage by 51.7% against 26.3% under the stronger placement. These results are empirical and structural, and we verify functional correctness rather than probing security. Building on this foundation, our future work will extend MAGE toward a formal masking-security result through share-preserving composition, fresh runtime randomness, and robust-probing verification.

VI. GRANT ACKNOWLEDGMENT

This work was supported by the U.S. Department of Energy, Office of Science, Basic Energy Sciences, under Award DE-SC0026398

VII. AI USE ACKNOWLEDGMENT

Generative AI and language-editing tools (e.g. ChatGPT, Claude, Grammarly) were used for grammar correction and language refinement. All edited content was reviewed and verified by the authors, who take full responsibility for the final manuscript.

REFERENCES

- [1] E. Trichina, “Combinational logic design for aes subbyte transformation on masked data,” *IACR Cryptol. ePrint Arch.*, vol. 2003, p. 236, 2003.
- [2] H. Groß, S. Mangard, and T. Korak, “Domain-oriented masking: Compact masked hardware implementations with arbitrary protection order,” *Cryptology ePrint Archive*, 2016.
- [3] D. Knichel, A. Moradi, N. Müller, and P. Sasdrich, “Automated generation of masked hardware,” *Cryptology ePrint Archive*, 2021.
- [4] L. Wu, Y. Fan, B. Preneel, W. Wang, and M. Wang, “Automated generation of masked nonlinear components: From lookup tables to private circuits,” in *International Conference on Applied Cryptography and Network Security*. Springer, 2024, pp. 319–339.
- [5] D. Knichel, P. Sasdrich, and A. Moradi, “Silver—statistical independence and leakage verification,” in *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 2020, pp. 787–816.
- [6] N. Müller and A. Moradi, “Prolead: A probing-based hardware leakage detection tool,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 311–348, 2022.
- [7] R. Sadhukhan, S. Saha, S. Paria, S. Bhunia, and D. Mukhopadhyay, “VALIANT: An EDA Flow for Side-Channel Leakage Evaluation and Tailored Protection,” *IEEE Transactions on Computers*, vol. 73, no. 2, pp. 436–450, 2024.
- [8] T. Mahfuz, S. Paria, T. Suha, S. Bhunia, and P. Chakraborty, “Polaris: Explainable artificial intelligence for mitigating power side-channel leakage,” in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2025, pp. 1–7.
- [9] E. Prouff and M. Rivain, “Masking against side-channel attacks: A formal security proof,” in *Advances in Cryptology – EUROCRYPT 2013*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 142–159.
- [10] B. J. Gilbert Goodwill, J. Jaffe, P. Rohatgi *et al.*, “A testing methodology for side-channel resistance validation,” in *NIST non-invasive attack testing workshop*, vol. 7, 2011, pp. 115–136.
- [11] P. Basu Roy, M. Nair, R. Sadhukhan, M. Alam, J. Knechtel, H. Pearce, D. Mukhopadhyay, O. Sinanoglu, and R. Karri, “Netlist whisperer: extensive analysis of circuit leakage using llms,” *Journal of Cryptographic Engineering*, vol. 15, no. 4, p. 22, 2025.
- [12] D. S. Koblah, D. M. Mehta, M. Hashemi, F. Ganji, and D. Forte, “Towards ai-driven optimization of robust probing model-compliant masked hardware gadgets using evolutionary algorithms,” *Cryptology ePrint Archive*, 2025.
- [13] M. Jenihhin, “<https://pld.ttu.ee/~maksim/benchmarks/>,” Benchmark circuits (ISCAS ’85). [Online]. Available: <https://pld.ttu.ee/~maksim/benchmarks/>
- [14] EPFL-LSI, “<https://www.epfl.ch/labs/lsi/page-102566-en.html/benchmarks/>,” The EPFL Combinational Benchmark Suite. [Online]. Available: <https://www.epfl.ch/labs/lsi/page-102566-en.html/benchmarks/>
- [15] MIT-LL, “<https://github.com/mit-ll/cep/>,” CEP: Common Evaluation Platform. [Online]. Available: <https://github.com/mit-ll/cep/>